

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-4:

- kod klasy populacji,
- wyświetlenie informacji o najlepszym osobniku
- parametr statyczny.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025

Kod klasy TPopulation

Populacja



id: ~~_____~~ unsigned int
candidates_count: unsigned int
candidates: Tcandidate[]
best_val: double

void **calculate()**
Tcandidate **get_best_candidate()**

Tcandidate [] – symbolicznie
zaznaczono wykorzystanie
„pojemnika”, nie konieczne
będzie to tablica...

Na dzisiejszych zajęciach implementujemy kod klasy bez
parametru id!

Populacja



id: unsigned int
candidates_count: unsigned int
candidates: Tcandidate[]
Best_val: double

void **calculate()**
Tcandidate **get_best_candidate()**

```
.cpp  TCandidate.cpp  TPopulation.h  TParam.h
G2024  TPopulation
1  #pragma once
2
3  #include <vector>
4  #include "TCandidate.h"
5
6  class TPopulation
7  {
8      unsigned int      candidated_count;
9      std::vector<TCandidate> candidates;
10     double             best_val = 0;
11
12     public:
13         TPopulation(unsigned int cands_count = 10);
14         void calculate();
15         TCandidate get_best_candidate();
16
17         void info();
18     };

```

```
.cpp  TCandidate.cpp  TPopulation.h  TParam.h
G2024  TPopulation
1  #pragma once
2
3  #include <vector>
4  #include "TCandidate.h"
5
6  class TPopulation
7  {
8      unsigned int      candidated_count;
9      std::vector<TCandidate> candidates;
10     double             best_val = 0;
11
12 public:
13     TPopulation(unsigned int cands_count = 10);
14     void         calculate();
15     TCandidate   get_best_candidate();
16
17     void         info();
18 };
```

Linia 9: wykorzystanie zmiennej typu vector.
W linii 3 dołączenie odpowiedniej biblioteki.

Linia 13: konstruktor z jednym domyślnym parametrem – liczbą osobników w populacji.

Linia 17: funkcja składowa do wyświetlania informacji na ekranie.

```
am.cpp TPopulation.cpp X
G2024 TPopulation TPopulation(unsigned int cand_s_c
1 #include <iostream>
2 #include <algorithm> // max
3
4 #include "TPopulation.h"
5
6 using namespace std;
7
8 TPopulation::TPopulation(unsigned int cand_s_count)
9 {
10     candidated_count = cand_s_count;
11
12     for (int i = 0; i < cand_s_count; i++) candidates.push_back({});
13     // for (int i = 0; i < cand_s_count; i++) candidates.push_back(TCandidate{});
14 }
15
16 void TPopulation::calculate()
17 {
18     double best_val = 0.0;
19
20     for (int i = 0; i < candidated_count; i++)
21     {
22         candidates[i].rate();
23         double val = candidates[i].get_mark();
24
25         if (i == 0) best_val = val;
26         else best_val = max(best_val, val);
27     }
28
29     this->best_val = best_val;
30 }
```

Konstruktor

Linia 12 lub 13: **utworzenie** obiektów klasy `Tcandidate`, będących elementami wektora `candidates` – parametru składowego naszej klasy.

TPopulation::calculate()

Linia 18: utworzenie zmiennej lokalnej do przechowywania chwilowej najlepszej wartości.

Linia 20 – 27: obliczanie w pętli wartości każdego osobnika populacji oraz wyznaczenie najlepszej wartości.

Linia 22 – 23: wywołanie funkcji z klasy `Tcandidate`.

Linia 26: skorzystanie z funkcji `max()` z biblioteki `algorithm` – dodana w linii 2.

Linia 29: zapisanie najlepszego wyniku do parametru składowego klasy.

```
32  TCandidate TPopulation::get_best_candidate()
33  {
34      int i = 0;
35
36      while (candidates[i].get_mark() != best_val) i++;
37
38      return candidates[i];
39  }
40
41  void TPopulation::info()
42  {
43      cout << "\n\n";
44      cout << "==== POPULATION #" << 0 << " =====\n";
45
46      for (int i = 0; i < candidated_count; i++)
47      {
48          cout << "== candidate#" << i << ": " << candidates[i].get_mark() << "\n";
49      }
50
51      cout << "=====\n\n";
52  }
53
```

Tpopulation::get_best_candidate()

Linia 34: zmienna pomocnicza.

Linia 36: zwiększanie w pętli wartości zmiennej lokalnej (indeksu tablicy) do momentu aż nie trafimy na osobnika (obiekt klasy Tcandidate) o najwyższej wartości w populacji.

Linia 38: zwrócenie obiektu osobnika o najwyższej wartości w populacji.

```
main.cpp x TCandidate.cpp TPopulation.h TParam.h
AG2024 (Globalny zasięg) main()
1 #include <iostream>
2 #include <cstdlib> // srand
3 #include <time.h>
4
5 #include "TPopulation.h"
6
7 using namespace std;
8
9
10 int main()
11 {
12     srand(time(0));
13
14     TPopulation pop(10);
15
16     cout << "Zaraz po utworzeniu obiektu klasy TPopulaiton";
17     pop.info();
18     cout << "\n\n";
19
20     cout << "Po wykonanych obliczeniach";
21     pop.calculate();
22     pop.info();
23     cout << "\n\n";
24
25     return 0;
26 }
```



```
Konsola debugowania programu Microsoft Visual Studio
Zaraz po utworzeniu obiektu klasy TPopulaiton

===== POPULATION #0 =====
== candidate#0: 0
== candidate#1: 0
== candidate#2: 0
== candidate#3: 0
== candidate#4: 0
== candidate#5: 0
== candidate#6: 0
== candidate#7: 0
== candidate#8: 0
== candidate#9: 0
=====

Po wykonanych obliczeniach

===== POPULATION #0 =====
== candidate#0: 90
== candidate#1: 9
== candidate#2: 101
== candidate#3: 64
== candidate#4: 106
== candidate#5: 90
== candidate#6: 24
== candidate#7: 54
== candidate#8: 90
== candidate#9: 16
=====
```

Linia 14: utworzenie populacji liczącej 10 osobników.

Linia 16 – 18: wyświetlenie informacji o populacji.

Linia 21: obliczenie wartości każdego osobnika populacji.

Linia 20 – 23: wyświetlenie informacji o populacji.

Zadanie

Zadanie



Konsola debugowania programu Microsoft Visual Studio

Zaraz po utworzeniu obiektu klasy TPopulaiton

```
===== POPULATION #0 =====  
== candidate#0: 0  
== candidate#1: 0  
== candidate#2: 0  
== candidate#3: 0  
== candidate#4: 0  
== candidate#5: 0  
== candidate#6: 0  
== candidate#7: 0  
== candidate#8: 0  
== candidate#9: 0  
=====
```

Po wykonanych obliczeniach

```
===== POPULATION #0 =====  
== candidate#0: 38  
== candidate#1: 25  
== candidate#2: 69  
== candidate#3: 107  
== candidate#4: 6  
== candidate#5: 36  
== candidate#6: 64  
== candidate#7: 18  
== candidate#8: 9  
== candidate#9: 10  
=====
```

Najlepszy osobnik:

```
=====  
==  
== gens count: 2  
== "x1" value: 10  
== "x2" value: 7  
==  
== rate: 107  
=====
```

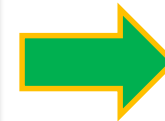
1. Wyświetlenie informacji o najlepszym osobniku

2. Dodanie parametru statycznego *_id* do klasy *TPopulation*

Parametr statyczny „Koło ratunkowe”

Efekt końcowy:

```
main.cpp  x  TKlasa.h
pomoc_populacja  (Globalny zasięg)
1  #include <iostream>
2  #include "TKlasa.h"
3
4  using namespace std;
5
6  int main()
7  {
8
9      TKlasa obiekt1{-3};
10     TKlasa obiekt2{0};
11     TKlasa obiekt3{5};
12
13     obiekt1.info();
14     obiekt2.info();
15     obiekt3.info();
16
17     cout << "=====\n\n";
18
19     obiekt1.set_vector_size(3);
20     obiekt3.set_init_val(2);
21
22     obiekt1.info();
23     obiekt2.info();
24     obiekt3.info();
25
26     return 0;
27 }
```



```
Konsola debugowania programu Microsoft Visual Studio
Obiekt klasy TKlasa o numerze: 1/3
Wartosci: -3, -2, -1, 0, 1,
Obiekt klasy TKlasa o numerze: 2/3
Wartosci: 0, 1, 2, 3, 4,
Obiekt klasy TKlasa o numerze: 3/3
Wartosci: 5, 6, 7, 8, 9,
=====
Obiekt klasy TKlasa o numerze: 1/3
Wartosci: -3, -2, -1,
Obiekt klasy TKlasa o numerze: 2/3
Wartosci: 0, 1, 2,
Obiekt klasy TKlasa o numerze: 3/3
Wartosci: 2, 3, 4,
```

Linia 19: zmiana dotyczy wartości parametru statycznego, dotyczy wszystkich obiektów klasy TKlasa!

Linia 20: zmiana dotyczy wartości parametru obiektu3

```
main.cpp*  TKlasa.h  X
pomoc_populacja  (Globalny zasięg)
1  #pragma once
2
3  class TKlasa
4  {
5      static unsigned int count;
6      static unsigned int vector_size;
7          int init_val;
8      unsigned int _id;
9
10 public:
11     TKlasa(int init_val);
12
13     void set_vector_size(unsigned int size);
14     void set_init_val(int init_val);
15     void info();
16 };
17
18
```

Linia 4-5: parametry statyczne!

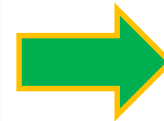
Zmienne statyczne

```
TKlasa.cpp  + X
pomoc_populacja  (Globalny zasięg)

1  #include "TKlasa.h"
2  #include <iostream>
3
4  using namespace std;
5
6  unsigned int TKlasa::count = 0;
7  unsigned int TKlasa::vector_size = 5;
8
9  TKlasa::TKlasa(int init_val)
10 {
11     TKlasa::count += 1;
12     _id = TKlasa::count;
13     // _id = this->count;
14
15     this->init_val = init_val;
16 }
17
18 void TKlasa::set_vector_size(unsigned int size)
19 {
20     vector_size = size;
21     // TKlasa::vector_size = size;
22 }
23
24 void TKlasa::set_init_val(int init_val)
25 {
26     this->init_val = init_val;
27 }
28
29 void TKlasa::info()
30 {
31     cout << "Obiekt klasy TKlasa o numerze: " << _id << "/" << this->count << endl;
32     cout << "Wartosci: ";
33
34     for (int i = 0; i < vector_size; i++)
35     {
36         cout << init_val + i << ", ";
37     }
38
39     cout << "\n\n";
40 }
```

Linie 6-7: podanie wartości początkowych parametrów statycznych.

```
main.cpp  x  TKlasa.h
pomoc_populacja  (Globalny zasięg)
1  #include <iostream>
2  #include "TKlasa.h"
3
4  using namespace std;
5
6  int main()
7  {
8
9      TKlasa obiekt1{-3};
10     TKlasa obiekt2{0};
11     TKlasa obiekt3{5};
12
13     obiekt1.info();
14     obiekt2.info();
15     obiekt3.info();
16
17     cout << "=====\n\n";
18
19     obiekt1.set_vector_size(3);
20     obiekt3.set_init_val(2);
21
22     obiekt1.info();
23     obiekt2.info();
24     obiekt3.info();
25
26     return 0;
27 }
```



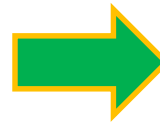
```
Konsola debugowania programu Microsoft Visual Studio
Obiekt klasy TKlasa o numerze: 1/3
Wartosci: -3, -2, -1, 0, 1,
Obiekt klasy TKlasa o numerze: 2/3
Wartosci: 0, 1, 2, 3, 4,
Obiekt klasy TKlasa o numerze: 3/3
Wartosci: 5, 6, 7, 8, 9,
=====
Obiekt klasy TKlasa o numerze: 1/3
Wartosci: -3, -2, -1,
Obiekt klasy TKlasa o numerze: 2/3
Wartosci: 0, 1, 2,
Obiekt klasy TKlasa o numerze: 3/3
Wartosci: 2, 3, 4,
```

Linia 19: zmiana dotyczy wartości parametru statycznego, dotyczy wszystkich obiektów klasy TKlasa!

Linia 20: zmiana dotyczy wartości parametru obiektu3

Tworzenie obiektów

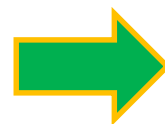
```
main.cpp  TKlasa.h
pomoc_populacja (Globalny zasięg)
1  #include <iostream>
2  #include <vector>
3  #include "TKlasa.h"
4
5  using namespace std;
6
7  int main()
8  {
9      vector <TKlasa> obiekty;
10
11      for (int i = 0; i < 3; i++)
12      {
13          obiekty.push_back(TKlasa{i});
14          // obiekty.push_back({ i });
15      }
16
17      for (int i = 0; i < 3; i++)
18      {
19          obiekty[i].info();
20      }
21
22      return 0;
23  }
```



```
Konsola debugowania programu Microsoft Visual Studio
Obiekt klasy TKlasa o numerze: 1/3
Wartosci: 0, 1, 2, 3, 4,
Obiekt klasy TKlasa o numerze: 2/3
Wartosci: 1, 2, 3, 4, 5,
Obiekt klasy TKlasa o numerze: 3/3
Wartosci: 2, 3, 4, 5, 6,
```

Tworzenie obiektów

```
main.cpp  X  TKlasa.h
pomoc_populacja  (Globalny zasięg)
1  #include <iostream>
2  #include <vector>
3  #include "TKlasa.h"
4
5  using namespace std;
6
7  int main()
8  {
9      vector <TKlasa> obiekty;
10
11      unsigned int count;
12      cout << "Podaj liczbe obiektow do utworzenia: ";
13      cin >> count;
14      cout << "\n";
15
16      for (int i = 0; i < count; i++)
17      {
18          obiekty.push_back({ i });
19      }
20
21      for (int i = 0; i < count; i++)
22      {
23          obiekty[i].info();
24      }
25
26      return 0;
27 }
```



```
Konsola debugowania programu Microsoft Visual Studio
Podaj liczbe obiektow do utworzenia: 5

Obiekt klasy TKlasa o numerze: 1/5
Wartosci: 0, 1, 2, 3, 4,

Obiekt klasy TKlasa o numerze: 2/5
Wartosci: 1, 2, 3, 4, 5,

Obiekt klasy TKlasa o numerze: 3/5
Wartosci: 2, 3, 4, 5, 6,

Obiekt klasy TKlasa o numerze: 4/5
Wartosci: 3, 4, 5, 6, 7,

Obiekt klasy TKlasa o numerze: 5/5
Wartosci: 4, 5, 6, 7, 8,
```

**Przykład ten przyda się na kolejnych zajęciach – warto zapisać kod!*

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-4:

- kod klasy populacji,
- wyświetlenie informacji o najlepszym osobniku
- parametr statyczny.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025